Detailed and Updated Summary:
Development of an Orchard Inspection Robot: A ROS-Based LiDAR-SLAM System with Hybrid A-DWA Navigation* *Sensors* 25 (2025) 6662

NECKER
Based on the original PDF + some clarifications

December 9, 2025

Abstract

This document presents a detailed analysis of the development of a mobile robot for autonomous inspection in orchards. A SLAM (Simultaneous Localization and Mapping) navigation system using LiDAR technology and multi-sensor fusion (LiDAR + IMU) is described. The kinematic model of the robot and URDF-based coordinate transformations within ROS are presented. The core of the navigation is a hybrid path planning strategy that combines an improved A^* algorithm (with adaptive weights) for global planning and the Dynamic Window Approach (DWA) for real-time obstacle avoidance. Experimental results demonstrate the system's effectiveness in terms of waypoint stopping precision and obstacle avoidance efficiency in unstructured environments.

Contents

1 Introduction

Automation of agricultural monitoring is essential for improving orchard management efficiency and reducing labor intensity. Autonomous mobile robots offer a promising solution compared to surveillance via drones or fixed stations, enabling detailed ground-level data collection. However, navigation in unstructured environments (such as orchards) presents significant challenges, particularly when GNSS signals are unreliable due to tree canopy shading.

This study proposes a system based on LiDAR SLAM, preferred for its high precision, real-time robustness, and adaptability to light conditions. The main innovations discussed include:

- Development of a mobile robot with tightly coupled multi-modal sensors (LiDAR and IMU).
- Kinematic modeling and coordinate transformations via URDF for ROS visualization.
- A hybrid planning strategy that merges an improved A^* (for global efficiency) and DWA (for local responsiveness).

2 Robotic System

The robot consists of a mobile platform with four independent driving wheels and differential steering. The main components include:

- **Chassis:** Equipped with double-wishbone independent suspension to minimize impact on the actuators.
- **Computing Unit:** Industrial computer (NP-6122-JH3) that processes data and sends commands to the chassis.
- **Sensors:**
  - IMU (WT901C-9) to measure velocity, acceleration, and attitude.
  - LiDAR (RS-LiDAR-16) for environmental data collection and map construction.
  - GNSS module and a gimbal camera on an elevating mast for image acquisition.

3 Kinematic Model and Transformations

The robot is described using the URDF (Unified Robotics Description Format) format, an XML format used in ROS to describe the physical characteristics of the robot. To handle the complex structure, the 3D model designed in SolidWorks was converted into a ROS-compatible URDF file.

3.1 Coordinate System Transformation

The transformation between two Cartesian coordinate systems follows Helmert's seven-parameter transformation model. The relationship between the vehicle coordinate system (C) and the LiDAR coordinate system (S) is fundamental. Given a rotation δ, φ, θ respectively around the x, y, z axes, and assuming a scale factor $k = 1$, the complete transformation is described by the following equation:

$$\begin{bmatrix} x_c \\ y_c \\ z_c \end{bmatrix} = \begin{bmatrix} x_t \\ y_t \\ z_t \end{bmatrix} + \begin{bmatrix} 1 & 0 & 0 \\ 0 & \cos\delta & -\sin\delta \\ 0 & \sin\delta & \cos\delta \end{bmatrix} \begin{bmatrix} \cos\varphi & 0 & \sin\varphi \\ 0 & 1 & 0 \\ -\sin\varphi & 0 & \cos\varphi \end{bmatrix} \begin{bmatrix} \cos\theta & -\sin\theta & 0 \\ \sin\theta & \cos\theta & 0 \\ 0 & 0 & 1 \end{bmatrix} \begin{bmatrix} x_s \\ y_s \\ z_s \end{bmatrix} \tag{1}$$

This transformation allows the fusion of LiDAR point cloud data with the vehicle's frame of reference.

3.2 Forward and Inverse Kinematics

The robot utilizes a four-wheel differential drive model. Defining V_L and V_R as the left and right wheel velocities, v_{cy} as the linear velocity of the chassis, and ω_c as the angular velocity, the kinematic relationships are:

Forward Kinematics (from wheels to chassis):
takes wheel velocities as input (e.g., revolutions per minute) and returns the chassis velocity.

$$\begin{bmatrix} v_{cy} \\ \omega_c \end{bmatrix} = \begin{bmatrix} \frac{1}{2} & \frac{1}{2} \\ -\frac{1}{c} & \frac{1}{c} \end{bmatrix} \begin{bmatrix} V_L \\ V_R \end{bmatrix} \tag{2}$$

Where c is the distance between the centerlines of the left and right wheels.

Inverse Kinematics (from chassis to wheels):
takes the desired chassis velocity and calculates how fast the wheels must move:

$$\begin{bmatrix} V_L \\ V_R \end{bmatrix} = \begin{bmatrix} 1 & -\frac{c}{2} \\ 1 & \frac{c}{2} \end{bmatrix} \begin{bmatrix} v_{cy} \\ \omega_c \end{bmatrix} \tag{3}$$

4 Architecture and Data Flow

The `move_base` node operates through a structured data flow within the ROS computational graph:

- **Sensory and State Inputs:** The system subscribes to LiDAR messages in the `sensor_msgs/LaserScan` and `sensor_msgs/PointCloud` formats for obstacle perception. It also subscribes to odometry data via `nav_msgs/Odometry` and coordinate transformations via the `tf` system to determine the robot's pose.
- **Control Outputs:** After processing optimal paths, the node publishes velocity commands in the `geometry_msgs/Twist` format on the `cmd_vel` topic, adjusting the linear and angular velocity of the chassis in real time.

4.1 Costmap Configuration

To represent obstacle information, the system maintains two distinct costmaps: a `global_costmap` for long-term planning and a `local_costmap` for reactive obstacle avoidance. Their configuration is hierarchical and divided into three layers:

1. Common Configuration (General Plugin)

This layer defines physical and sensory parameters shared by both maps:

- `footprint`: Defines the physical area occupied by the robot, essential for computing collisions and ensuring that the robot passes through narrow spaces.
- `observation_sources`: Set to `scan`, it specifies that the primary source for updating the map is the LiDAR point cloud.
- `obstacle_range` (**2.5 m**): Defines the maximum distance within which the sensor inserts new obstacles into the costmap. Obstacles detected beyond this distance are ignored to keep the map clear of distant noise.
- `raytrace_range` (**3.0 m**): Defines the maximum distance for the "raytracing" operation (clearing space). If the sensor scans an area up to 3 meters and finds nothing, the system removes any obstacles previously marked in that space. It is set to a higher value than `obstacle_range` to ensure consistent free-space management.
- `min_obstacle_height` (**0.25 m**) and `max_obstacle_height` (**0.35 m**): These height filters are critical for orchard operations. They allow filtering out terrain irregularities and low grass (below 25 cm) and focusing avoidance on trunks and relevant obstacles that could hit the chassis, avoiding false positives.
- `inflation_radius`: The expansion radius around obstacles that creates a safety buffer zone. This parameter was subject to experimental optimization (optimal result at 0.4 m).

2. Global Costmap Configuration

Used by the global planner (A^*), this map covers the entire operating environment:

- `global_frame`: Set to `map`, referencing the fixed world coordinate system.
- `robot_base_frame`: Set to `base_link`, referencing the geometric center of the robot.
- `update_frequency`: **1 Hz**. Since the global map primarily contains static obstacles, a low update frequency is sufficient and saves computational resources.

3. Local Costmap Configuration

Used by the local planner (DWA), this map is a moving window (`rolling window`) centered on the robot:

- **Dimensions and Resolution:** Configured as a window of **10 m** $\times$ **10 m** with a resolution of **0.5 m**. This relatively low resolution reduces the computational load, allowing DWA to calculate complex trajectories rapidly.
- `publish_frequency`: **2 Hz**. The local map is updated and published more frequently to respond to immediate dynamic changes in the surrounding environment.
- The reference frames are aligned with the global configuration to ensure consistency between the two planners.

5 Path Planning Strategy (Hybrid A^*-DWA)

To guarantee efficient and safe autonomous navigation in an unstructured environment such as an orchard, a hybrid strategy was adopted. This combines an improved global A^* algorithm, to define the optimal long-range trajectory, with the Dynamic Window Approach (DWA) for local kinematics management and avoidance of unexpected obstacles.

5.1 Improved A^* Algorithm

The standard A^* algorithm evaluates nodes using the cost function $f(n) = g(n) + h(n)$, where $g(n)$ is the actual cost from the starting node and $h(n)$ is the estimated heuristic cost toward the goal. In this study, the Euclidean distance metric is used to compute both values. However, the standard approach assigns equal weights to $g(n)$ and $h(n)$, which can lead to the expansion of an excessive number of candidate nodes, reducing search efficiency.

To optimize this process, an adaptive weight function $w(n)$ applied to the heuristic component was introduced. The goal is to favor wide exploration in the initial stages (Dijkstra-like behavior) and accelerate convergence toward the goal in the final stages (Greedy Best-First Search-like behavior). The modified cost function is defined as:

$$f(n) = g(n) + \left\{ \sin^2 \left[\frac{\pi}{2} \left(1 - \frac{d}{D} \right) \right] + \frac{1}{2} \right\} h(n) \tag{4}$$

Where:

- d represents the Euclidean distance from the starting node to the current node.
- D represents the total estimated distance from the starting node to the goal node.

The term in curly brackets represents the dynamic weight. This weight varies sinusoidally, ensuring a smooth transition within the $[0.5, 1.5]$ interval. The bounds were determined empirically: a weight below 0.5 would result in excessive exploration, while a weight above 1.5 would risk compromising path optimality for computation speed. The sinusoidal function ensures that the weight is close to 0.5 when $\frac{d}{D}$ is small (start of the path) and tends to 1.5 when $\frac{d}{D}$ approaches 1 (near the goal).

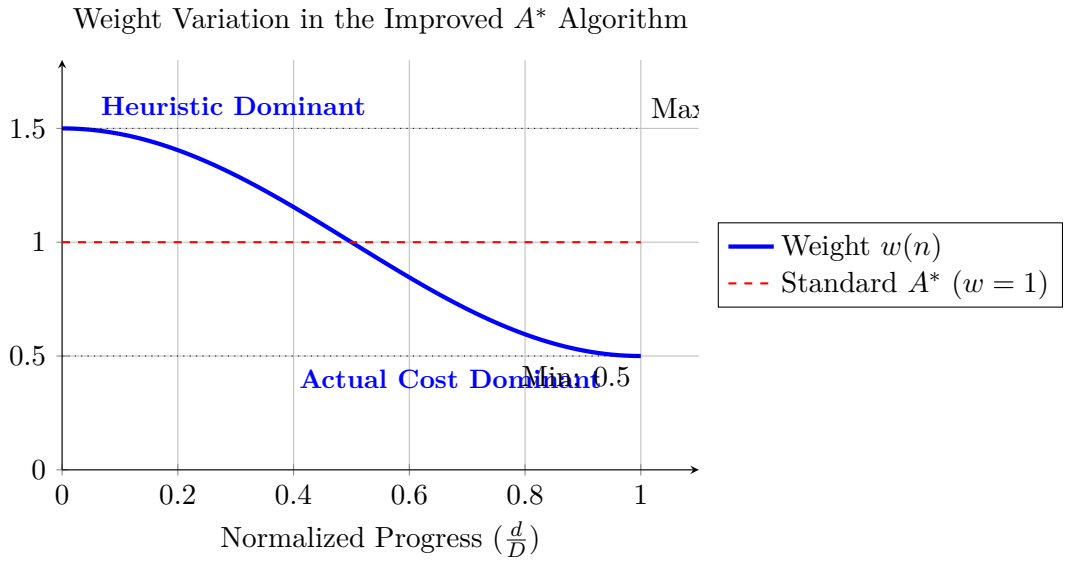

Figure 1: Behavior of the adaptive weight function. The weight decreases sinusoidally from 1.5 to 0.5 as the robot approaches the objective ($d \to D$), shifting from a greedy approach to a more accurate one.

5.2 DWA Algorithm (Local Planner)

While A^* defines the static path, the Dynamic Window Approach (DWA) handles real-time control. The algorithm samples multiple velocity trajectories (v, ω) in the robot's velocity space, considering kinematic and dynamic constraints (maximum velocity, acceleration). In practice, based on the current situation, there are available trajectories to choose from (or from a robotic standpoint, operations to execute such as turning the wheels or increasing angular velocity). The optimal trajectory is selected by maximizing the following objective function G:

$$G(v, \omega) = \sigma[\alpha \cdot \text{head}(v, \omega) + \beta \cdot \text{dist}(v, \omega) + \gamma \cdot \text{vel}(v, \omega)] \tag{5}$$

The components of the function are:

- $\text{head}(v, \omega)$: Evaluates how much a trajectory aligns the robot with the direction of the local target (if the trajectory causes the robot to face away from the target, the score is very low).
- $\text{dist}(v, \omega)$: Represents the minimum distance to the nearest obstacle; higher values indicate greater safety (distant obstacles).
- $\text{vel}(v, \omega)$: Measures how much a trajectory helps finish the path sooner (rewards higher linear velocities to ensure operational efficiency).
- σ: Normalization coefficient to make the three components comparable.

The weight coefficients (α, β, γ) were calibrated specifically for the orchard environment. Through extensive simulations, the parameters were fixed at $\alpha = 0.6$, $\beta = 0.3$, and $\gamma = 0.1$.

This configuration assigns the highest priority (α) to orientation toward the goal to ensure progress, followed by safety relative to obstacles (β). The velocity weight (γ) is kept low to avoid aggressive movements or instability on uneven terrain, which could compromise the quality of images collected during inspection.

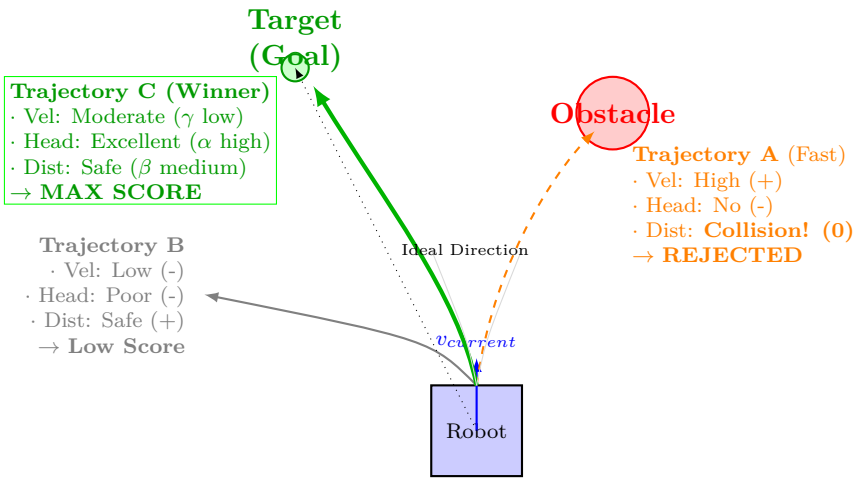

Figure 2: Graphical representation of the Dynamic Window Approach (DWA). The green box (Trajectory C) highlights the optimal choice that balances direction and safety.

5.3 Hybrid Method and Integration

The integration of the two algorithms occurs hierarchically, bridging the gap between static map planning and dynamic navigation. The node and target selection process proceeds as follows:

Node Selection (Global Level)

The map is discretized into a grid with a resolution of **0.5 m** (as defined in the Costmap configuration). Each grid cell represents a candidate node.

- The A^* algorithm evaluates these nodes and selects the sequence that minimizes the cost function $f(n)$.
- The chosen nodes form the "global path" (red line in visualizations), which is geometrically optimal but does not account for the robot's kinematics.

Target Selection (Local Level)

The DWA local planner does not point directly to the final destination, but follows the global path one step at a time.

- **Guidance Node Extraction:** The system analyzes the global path and selects a node located at a certain distance ahead of the robot (`lookahead distance`). This node becomes the temporary **Local Target**.
- **Trajectory Computation:** The DWA generates the fan of possible trajectories and chooses the one that maximizes the function $G(v, \omega)$, pointing toward the Local Target and avoiding obstacles detected in real time by the sensors.
- As the robot advances, the Local Target is updated by scrolling along the nodes of the global path, guiding the robot smoothly to its destination.

6 Simulation Results

To validate the proposed approach, simulations were conducted in a MATLAB R2022b environment using 2D grid maps.

6.1 Global Planning Comparison

The improved A^* algorithm was compared with the standard A^* algorithm and Dijkstra's algorithm in terms of computation time, path length, and number of explored nodes. The results are summarized in Table 1.

Table 1: Performance comparison of global planning algorithms

Algorithm	**Time (s)**	**Path Length (m)**	**Nodes Traversed**
Standard A^*	10.5	44.32	427
Dijkstra	18.7	38.79	684
Improved A^*	**4.3**	**38.79**	**374**

As highlighted by the data, the improved A^* drastically reduces planning time (from 10.5 s to 4.3 s compared to the standard) and the number of expanded nodes, while maintaining an optimal path length comparable to Dijkstra's (38.79 m). This demonstrates superior search efficiency due to the adaptive heuristic function.

6.2 Local Planning Comparison

For local planning, the DWA method was compared with the Artificial Potential Field (APF) algorithm, a classic reactive method. Simulations showed that while both avoid obstacles, the path generated by DWA is significantly smoother.

- **DWA trajectory length:** 15.32 m.
- **Stability:** Unlike APF, which can suffer from local minima and oscillations, DWA showed consistent and gradual variations in angular velocity.

This "smoothness" translates into a stable motion posture and a reduction in robot jitter, critical factors for ensuring that images acquired by the camera during movement do not turn out blurry or unusable.

7 Field Experimentation

Tests were conducted at the Yangzijin campus of Yangzhou University in an unstructured environment.

7.1 Waypoint Stopping Test

The robot was required to stop for 5 seconds at 4 designated waypoints. The cruising speed was set at 0.8 m/s.

- **Mean Lateral Error:** 0.163 m (Max: 0.273 m).
- **Mean Longitudinal Error:** 0.282 m (Max: 0.526 m).
- **Mean Orientation Error:** 2.9°.
- **Reaction Times:** Average braking 0.46 s, restarting 0.64 s.

Despite deviations due to terrain irregularities, precision was deemed sufficient for inspection operations, which require stopping within 1 meter.

7.2 Obstacle Avoidance Test

The impact of the `inflation_radius` parameter on obstacle avoidance performance was evaluated (Table 2).

Table 2: Obstacle avoidance test results with various inflation radii

Radius (m)	Trajectory Confusion	Collision	Min. Distance (m)	Outcome
1.0	Yes	No	1.284	Failed (Disoriented)
0.8	No	No	0.826	Wide steering radius
0.6	No	No	0.643	Sharp steering
0.4	**No**	**No**	**0.375**	**Optimal**
0.3	No	Yes	0	Collision

The optimal value proved to be 0.4 m, which balances safety with the ability to navigate in tight spaces without causing planner confusion.

8 Bibliographical References for Technical In-Depths

Since the original paper focuses on the specific application and experimental results, the detailed theoretical explanations present in this document (specifically the internal workings of DWA, ROS costmap logic, and spatial discretization) reference standard mobile robotics literature. The primary sources for the appended concepts are listed below:

Dynamic Window Approach (DWA)

The logical explanation of DWA as a trajectory sampling process (represented in the fan diagram) and the metaphor of "voting" via an objective function reference the original paper that introduced the algorithm:
D. Fox, W. Burgard, and S. Thrun, "The Dynamic Window Approach to Collision Avoidance", in *IEEE Robotics & Automation Magazine*, 1997.
[IEEE Xplore] | [PDF - CMU]

ROS Navigation Architecture (move_base)

The specific concepts of a *Rolling Window* for the Local Costmap and the technical management of inflation layers (*Inflation Layer*) are described in the paper presenting the standard ROS navigation stack used in this study:
E. Marder-Eppstein, E. Berger, T. Foote, B. Gerkey, and K. Konolige, "The Office Marathon: Robust Navigation in an Indoor Office Environment", in *International Conference on Robotics and Automation (ICRA)*, 2010.
[IEEE Xplore] | [ROS Wiki Documentation]

Occupancy Grids and Discretization

The mathematical theory behind the discretization of space into "nodes" (Occupancy Grid Maps) and the probabilistic definition of free/occupied space stems from the reference text:
S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. MIT Press, 2005.
[Official Site]

References

[1] Jiwei Qu, Yanqiu Gu, Zhinuo Qiu, Kangquan Guo, Qingzhen Zhu (2025).
Development of an Orchard Inspection Robot: A ROS-Based LiDAR-SLAM System with Hybrid A-DWA Navigation.*
Sensors, 25(21), 6662.
https://doi.org/10.3390/s25216662